

LINFO2132 - Project Phase 2 (Parser) Report

1. Context-Free Grammar (CFG)

The parser successfully implements the syntax analysis for the language based on the following grammar rules. The grammar covers variable declarations, control structures, and mathematical/relational expressions:

Program ::= Statement*
Statement ::= Declaration | IfStatement | WhileStatement | ForStatement | Block | Expression
";"

Declaration ::= TYPE IDENTIFIER ("=" Expression)? ";"

IfStatement ::= "if" "(" Expression ")" Block ("else" Block)?

WhileStatement ::= "while" "(" Expression ")" Block

ForStatement ::= "for" "(" IDENTIFIER ";" Expression "->" Expression ";" Expression ")" Block

Block ::= "{" Statement* "}"

Expression ::= Term (("+" | "-" | "==" | "!=" | ">" | "<" | ">=" | "<=") Term)*

Term ::= Factor (("*" | "/") Factor)*

Factor ::= INT | IDENTIFIER

2. Implementation Details

- **AST Structure:** For each major grammatical rule, a specific AST Node class was created (e.g., VarDeclNode, IfNode, WhileNode, ForNode, BinOpNode). The parser builds and links these nodes hierarchically.
- **Operator Precedence:** The parser correctly handles operator precedence by splitting the parsing of mathematical expressions into `parseExpression()` (for addition, subtraction, and relational operators) and `parseTerm()` (for multiplication and division).
- **Handling the - > Operator in For-Loops:** Since the lexer parses -> as two separate symbols (- and >), the parser specifically anticipates this sequence in the `parseForStatement()` method to ensure correct syntax tree generation without modifying the base lexer logic.

3. Unsupported Features

All primary requirements for Phase 2, including variable declarations, if-statements, while-loops, for-loops, and basic expressions, have been fully implemented and tested. No major features from the core requirement were left unhandled.